

Course project

David Jao

October 8, 2025



Further topics

Topics covered in the course:

- Combinatorial games ✓
- Strategic games ✓
- Cooperative games ←
- Auction theory ←
- Mechanism design ←

Course project

The course project will be on a separate topic: *partisan games*.

Definition

A *partisan* game is a non-imparital combinatorial game.

Examples:

- Chess
- Go
- Tic-tac-toe

Chess but captures are forced

The project is to write a computer program that plays *Chess but captures are forced*.

WARNING

There are many chess variations which go by the names Antichess, Losing Chess, Giveaway Chess, Suicide Chess, etc. Most of these variants are NOT the same as our variant.

Our variant of chess is identical to standard chess, except with one additional rule:

- If the player to move has a legal chess move which captures an opponent's piece, then the player to move must make a legal chess move which captures an opponent's piece.

Chess but captures are forced

Properties of this variant:

- Every single variant game is a valid chess game (but the reverse is not true).
- The result (win/lose/draw) of a variant game is the same as the result of the corresponding chess game. (You are not “trying” to lose.)
- You still have to keep track of the corner cases in chess:
 - Castling
 - Draw by repetition
 - 50-move rule

There are some subtle differences from normal chess, so you'll have to invent some novel strategy elements.

Basic strategy

Chess, but Captures are Forced: <https://www.youtube.com/watch?v=NfjnsFHPdFU>

- Use piece sacrifices to force your opponents to kamikaze their pieces, or expose their king.
- Checkmate the undefended king.

Using forced moves makes it easier to perform deep searches.

Techniques for analyzing partisan games

Some useful techniques:

- Minimax strategy
- Alpha-beta pruning

There's more, but the above is enough for this project.

Consult other classes at Waterloo on artificial intelligence for a more thorough treatment (ECE457[ABC], CS48[056], etc.)

Algorithms explained — minimax and alpha-beta pruning:

<https://www.youtube.com/watch?v=1-hh51ncgDI>

Minimax

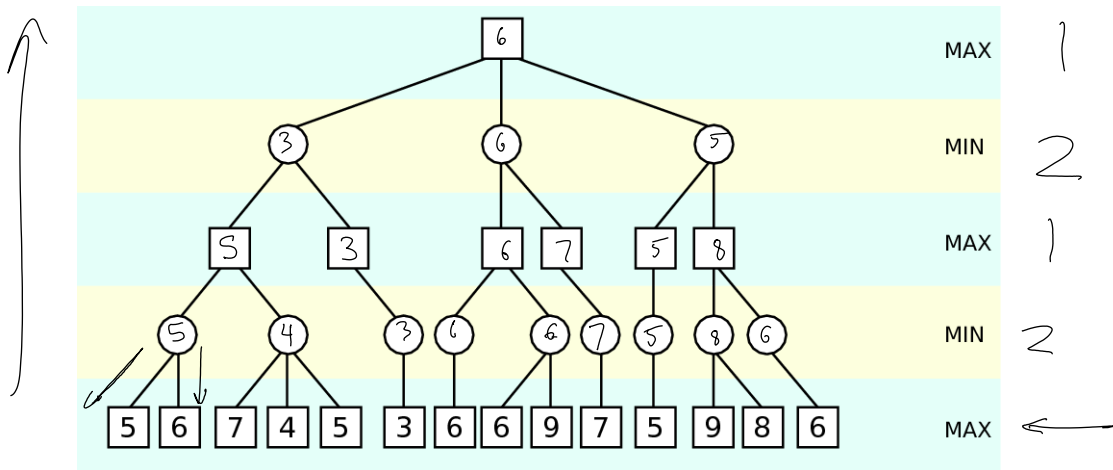
Suppose we know the payoff values of each outcome.

- Form the game tree of all positions and outcomes.
- Label each leaf node with its payoff value.
- At each non-leaf node, label the node with
 - The *maximum* payoff of its children, if you are moving;
 - The *minimum* payoff of its children, if your opponent is moving.
- Choose at each node from among the choices with maximum payoff.

Minimax example

(From https://commons.wikimedia.org/wiki/File:AB_pruning.svg)

Square nodes denote MAX, round nodes denote MIN



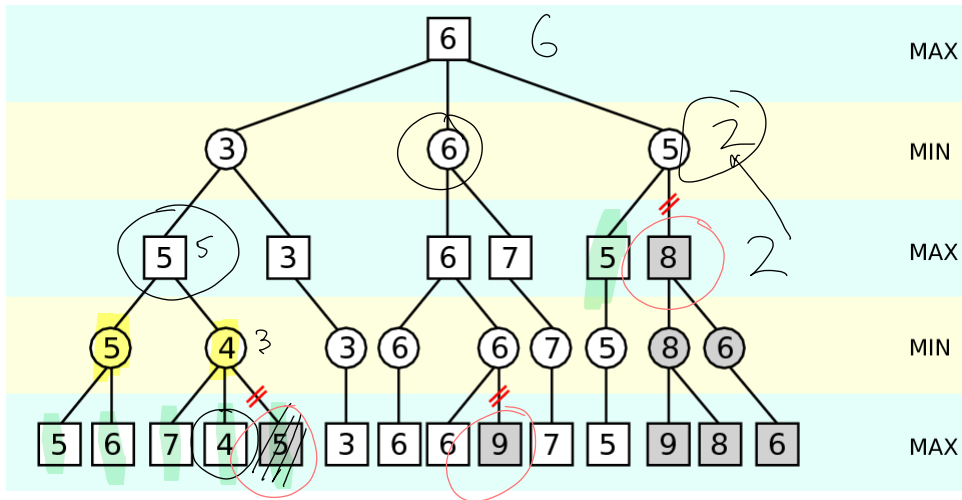
Evaluation functions

Drawing the entire game tree is impractical for chess.

- **Estimate** the payoff value of each position using a static evaluation function.
- Having a good (and fast!) evaluation function is critical.
- If you can't provide a good estimate, searching one or two moves deeper usually helps a lot. But you also have to know when to stop.

Alpha-beta pruning

Idea: stop searching branches that are estimated to be worse than what you already know.



Program requirements

In order to allow students to choose their programming languages and environments freely, while still making tournament administration possible:

- You may use any programming language and libraries.
- Your program must be provided as source code.
- Your program must compile and run on Linux (Ubuntu 24.04).
- Your program must be compatible with the WinBoard / XBoard protocol (<http://hgm.nubati.net/newspecs.html>).

Tournament format

Grading scheme:

- Programs will play each other in either a round-robin or Swiss tournament with n rounds ($n \approx 5$), subject to change depending on number of teams.
- You get 60% (i.e. 15% out of 25%) just for submitting a program that correctly and non-degenerately participates in the tournament.
- The remaining marks will be distributed as a function of performance in tournament play.
- Time control: 1 minute per side (subject to change), i.e. 1+0

Project administration

- Due date: December 15. NO INCOMPLETES
- You may form a team of at most 3 people.
 - You are responsible for forming your own teams. You may use Piazza to form teams.
 - You must choose a name to identify your team.
 - You must tell me your team membership and name by November 1.
- You may use existing code (e.g. from the internet) as a base for your own code — as long as that existing code isn't specifically for this variant.
 - You must have the legal right to use the code.
 - You must cite any external code that you use.
- One CPU thread only (no multi-threading, no multi-processing)
- No hacking or attempts at hacking.